МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационной безопасности

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Защита компьютерной информации»

Тема: УПРАВЛЕНИЕ ДОСТУПОМ К ФАЙЛОВОЙ СИСТЕМЕ NTFS С ПОМОЩЬЮ СПИСКОВ КОНТРОЛЯ ДОСТУПА

Студентка гр. 3314 _____ Семенихина В.А.

Преподаватель _____ Новакова Н.Е.

Санкт-Петербург

2026

Цель работы.

Знакомство с механизмами управления доступом к файловой системе, основанными на списках контроля доступа (Access Control List - ACL). Работа выполнена в операционной системе ALT Linux (ALT Server), где аналогичные механизмы реализованы списками контроля доступа POSIX ACL.

Задание:

1. Подготовка: создать на виртуальной машине пользователей (имена на латинице) student1, student2, teacher; создать группы students, lishentsy (транслитерация)
2. Упражнение 1: создать файл Text1.txt и каталоги Dir1/Dir2 с файлами Text11.txt и Text12.txt; изучить права доступа к Text1.txt; назначить права пользователю student1 и группам students и lishentsy, проверить результат.
3. Упражнение 2: дать student1 только возможность управления ACL файла, от его имени восстановить возможность изменения файла.
4. Упражнение 3: настроить для student1 права на каталог Dir1, проверить наследование, определить действующие права.
5. Упражнение 4: проверить отключение наследования, преобразование и удаление записей ACL, смену владельца, повторное включение наследования.

Выполнение.

Лабораторная работа выполнена на виртуальной машине VirtualBox под управлением ALT Server (ALT Linux).

Подготовка

На виртуальной машине созданы пользователи student1, student2, teacher и группы students, lishentsy. (на английском, потому что я против русских названий файлов, пользователей и т.д., а также потому что в Linux имена пользователей и групп принято записывать латиницей и строчными буквами. Пользователей и

группы я создавала через «Центр управления системой», всё остальное выполнялось через командную строку, в том числе из-за того, что как пользователь Администратор (root) в систему зайти нельзя, да и менять пользователей в условиях командной строки проще)

Пункт 4 (устройство D:): требование методики о создании отдельного раздела в данной работе не выполнялось. В Linux нет букв дисков, все разделы входят в единое дерево каталогов, поэтому роль корневого каталога D:\ выполняет корень файловой системы /, и все файлы лабораторной работы созданы в нём.

Упражнение 1 - Назначение прав доступа пользователей и групп на файл

Пункт 1. Создание файлов и каталогов

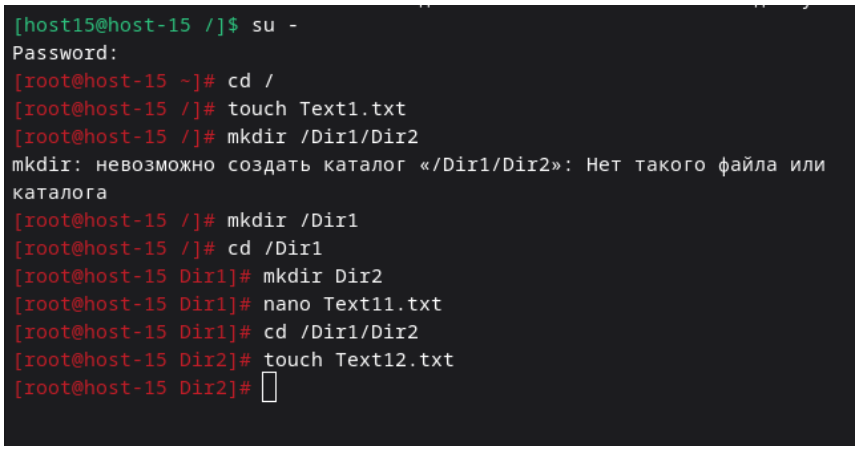


Рис. 1 - Создание каталогов Dir1, Dir2 и файлов Text11.txt, Text12.txt

Командой `su` – выполнен вход под root, после чего при помощи `touch Text1.txt` был создан текстовый файл “Text1.txt”. Затем командами `mkdir /Dir1` и `mkdir Dir2` была создана иерархия каталогов, Text11.txt создан в Dir1 редактором `nano`, Text12.txt - командой `touch` в Dir2.

(Различие touch и nano в том, что первый просто создаёт файл, а nano позволяет добавить в файл содержимое)

По заданию файлы должны содержать произвольный текст, поэтому в Text1.txt и Text12.txt записан текст «Test», а в Text11.txt текст введён в редакторе nano.

Пункт 2.1. Права доступа к Text1.txt

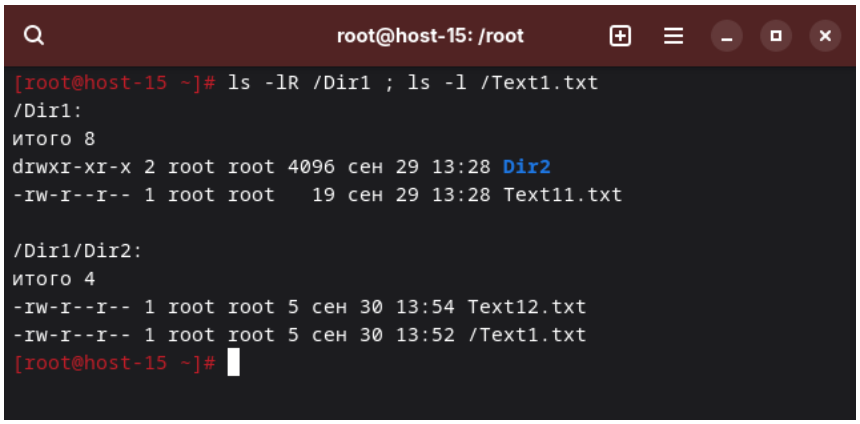


Рис. 2 - Права доступа к файлам и каталогам по умолчанию

Командой `ls -lR /Dir1 ; ls -l /Text1.txt` получены права объектов. Файл Text1.txt имеет права `-rw-r--r--`, root (владелец файла) имеет права на чтение и запись, группа root и все остальные имеют право на чтение. Каталог Dir2 имеет права `drwxr-xr-x`. (d - это обозначение каталога, остальные буквы также, как и в первом случае бьются по тройкам, буква x обозначает разрешение “входа” в каталог)

Таким образом, доступ к Text1.txt предоставлен так: владельцу root - чтение и запись (`rw-`), группе root - чтение (`r--`), всем остальным - чтение (`r--`). Других пользователей и групп с отдельными правами у файла нет.

Из источника 1:

r - владелец файла имеет разрешение на чтение (например, выполнить cat filename и увидеть содержимое файла)

w - владелец может записывать в файл (например, сделать echo "Hello world" > filename или изменить его в текстовом редакторе)

x - владелец может выполнять файл как программу или скрипт (например, запустить его с помощью команды ./filename, как и любую другую команду)

Пункт 2.2. Проверка возможностей student1

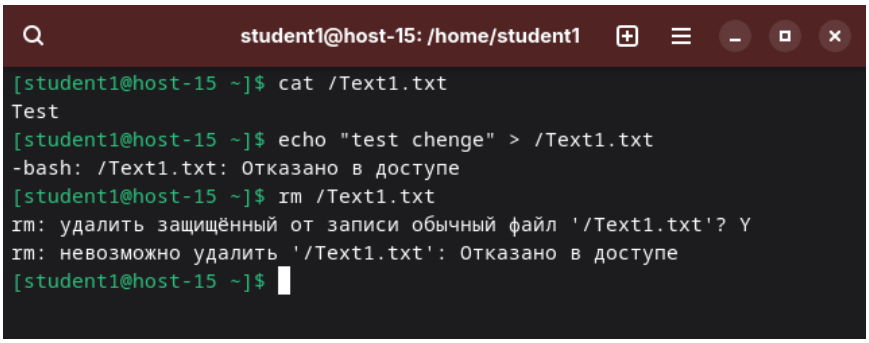


Рис. 3 - Попытки student1 прочитать, изменить и удалить Text1.txt

Под пользователем student1 команда `cat` прочитала файл, а запись через `echo` и удаление через `rm` завершились ошибкой «Отказано в доступе». Запись запрещена, потому что бит w есть только у владельца. Удаление запрещено по другой причине: в Linux оно определяется правами на каталог, содержащий файл, а не правами на сам файл, а у student1 нет права записи в каталог /. (echo может перезаписывать файл (>) и добавлять строки в файл (>>))

(При выполнении `rm` система сначала запросила подтверждение удаления файла, защищённого от записи; после ответа «Y» удаление всё равно завершилось отказом.)

Пункт 2.3. Право редактирования для student1

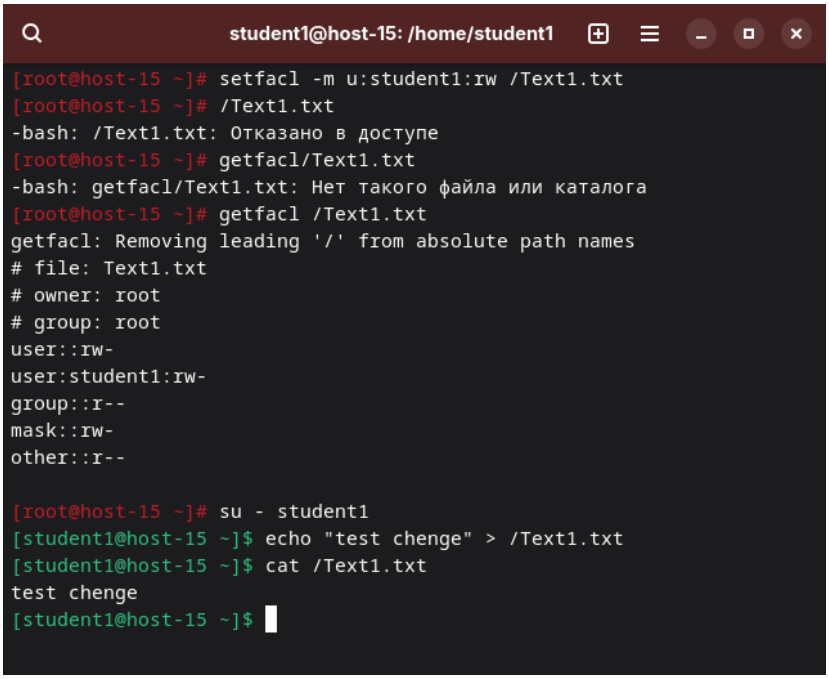


Рис. 4 - Назначение student1 прав rw на Text1.txt

Командой `setfacl -m u:student1:rw /Text1.txt` в ACL файла добавлена запись пользователя. Вывод `getfacl` показывает новую строку `user:student1:rw-` и маску `mask::rw-`. После входа под student1 запись в файл и чтение прошли успешно. Права выданы точечно: владелец и группа файла не менялись.

Пункт 2.4. Группа students

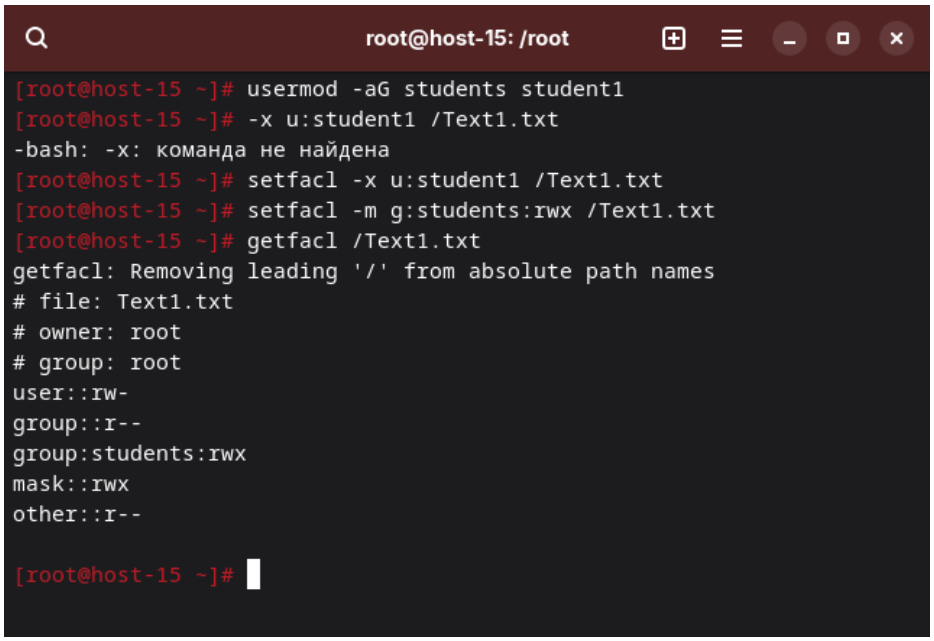


Рис. 5 - Добавление student1 в группу students и назначение ей прав rwx

Командой `usermod -aG students student1` пользователь добавлен в группу, затем персональная запись student1 удалена (`setfacl -x u:student1 /Text1.txt`), чтобы доступ определялся только группой, и группе выданы права `setfacl -m g:students:rwx /Text1.txt`. В выводе `getfacl` появились `group:students:rwx` и `mask::rwx` (Из источника 3: “Маска ACL позволяет принудительно ограничить действие всех назначенных для ФО правил ACL. Маска содержит разрешенные для ФО операции. По умолчанию маска ФО создается автоматически при создании для ФО первого правила ACL, и изменяется автоматически по мере добавления новых правил, представляя собой объединение всех назначенных правил ACL (то есть маска, создаваемая по умолчанию, разрешает все операции, заданные правилами ACL, и не ограничивает работу правил ACL).”), Маска выступает в

роли “фильтра” безопасности. Даже если конкретному пользователю или группе через ACL выданы полные права (rwx), но в маске указано только чтение (r--), итоговые права этого пользователя будут урезаны до чтения (r--). Работоспособность проверена на Рис. 7.

Отдельной проверки только с группой students на рисунках нет: запись в файл проверялась уже после добавления student1 в группу lishentsy (рис. 7). Так как lishentsy даёт только `r--`, запись могла пройти лишь за счёт прав `rwx` группы students, то есть право группы действует. Удаление файла по условию задания не проверялось; в любом случае оно определяется правами на каталог `/`, а не на файл.

Пункт 2.5. Группа lishentsy

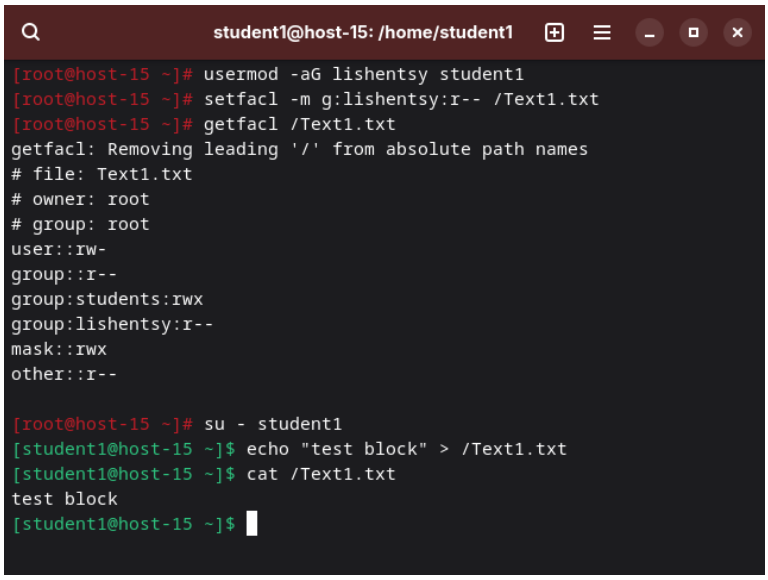


Рис. 6 - Добавление student1 в группу lishentsy и назначение ей прав r--

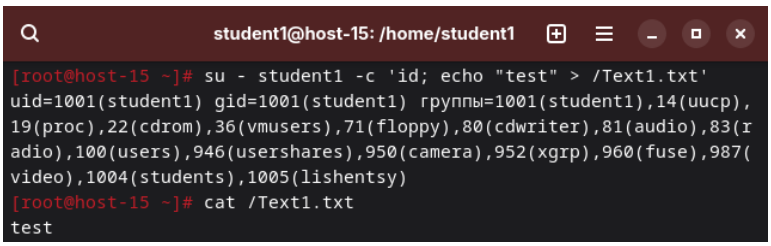


Рис. 7 - Членство student1 в группах и запись в файл

student1 добавлен в группу lishentsy, группе выдано только чтение: `setfacl -m g:lishentsy:r-- /Text1.txt` (рис. 6). Несмотря на это,

запись от имени student1 прошла успешно. Команда `id` (Рис. 7) подтверждает, что student1 состоит сразу в двух группах: students (1004) и lishentsy (1005).

Вывод: Если пользователь подпадает под несколько групповых записей, доступ предоставляется, когда его даёт хотя бы одна из них. Поэтому запись lishentsy:r-- не отменила права students:rwx. Ограничить доступ можно только отсутствием разрешающих записей или персональной записью пользователя (например, user:student1:---), которая проверяется раньше групповых.

Порядок проверки доступа по ACL (источник 7): если пользователь - владелец файла, действует запись владельца; иначе, если есть запись именованного пользователя, действует она (с учётом маски); иначе просматриваются записи всех групп пользователя, и доступ разрешается, если хотя бы одна из них даёт нужное право (с учётом маски) или запрет; иначе действует запись «остальные». Именно поэтому запись lishentsy:r-- ничего не запретила.

Упражнение 2 - Работа с детализированным представлением ACL

Пункт 1. Только управление ACL

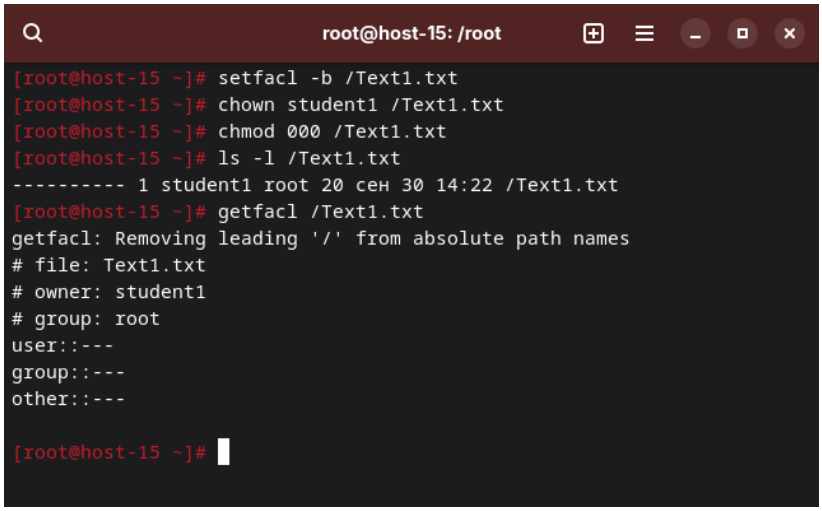


Рис. 8 - Передача владения Text1.txt пользователю student1 и обнуление прав

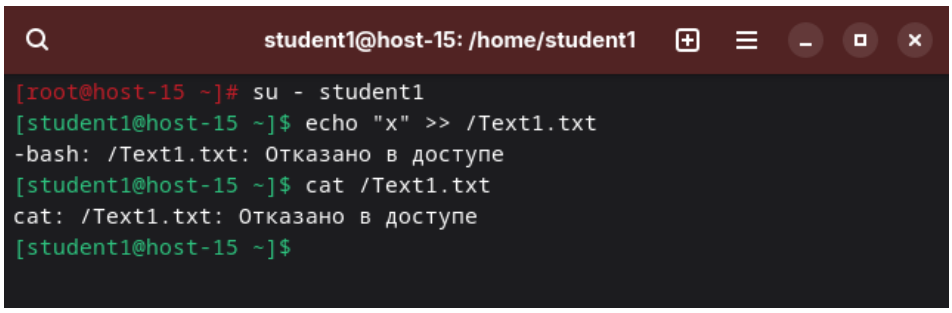


Рис. 9 - Проверка: student1 не может ни прочитать, ни изменить файл

В Linux право изменять ACL принадлежит только владельцу объекта и root (это из источника 6), и отдельным правом его выдать нельзя. Поэтому выполнены команды: `setfacl -b /Text1.txt` (удалены все расширенные записи), `chown student1 /Text1.txt` (student1 стал владельцем) и `chmod 000 /Text1.txt` (все права на содержимое сняты). Результат на рисунке 8: `ls -l` показывает `---------- student1 root`, а `getfacl` - нулевые права для владельца, группы и остальных. Проверка (рис. 9): student1 не может ни дописать, ни прочитать файл, то есть у него остаётся только возможность управлять правами.

Пункт 2. Восстановление доступа от имени student1

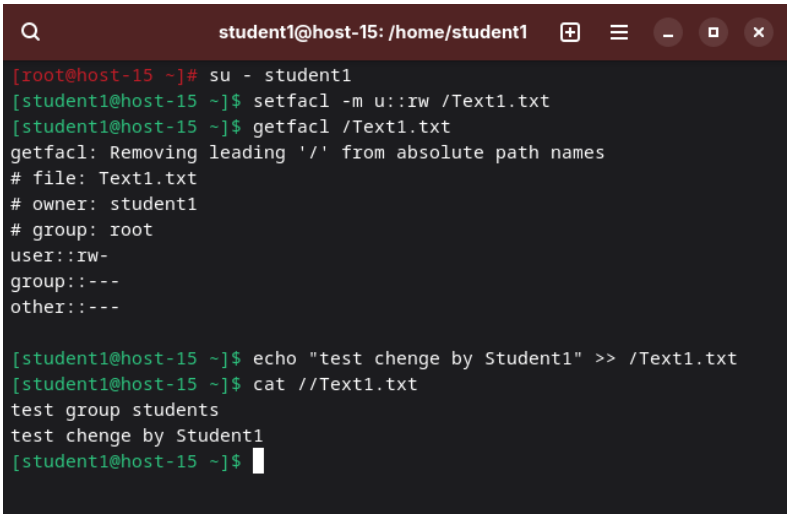


Рис. 10 - student1 возвращает себе права на файл командой setfacl

Под пользователем student1 выполнена команда `setfacl -m u::rw /Text1.txt`, изменяющая запись владельца. `getfacl` показывает `user::rw-`, дописывание строки в файл и чтение выполняются без участия root.

Владелец, не имея прав на содержимое, может сам себе их выдать, потому что управление ACL привязано к владению файлом.

Упражнение 3 - Использование наследования, определение эффективных прав доступа

Пункт 1. Права student1 на каталог Dir1

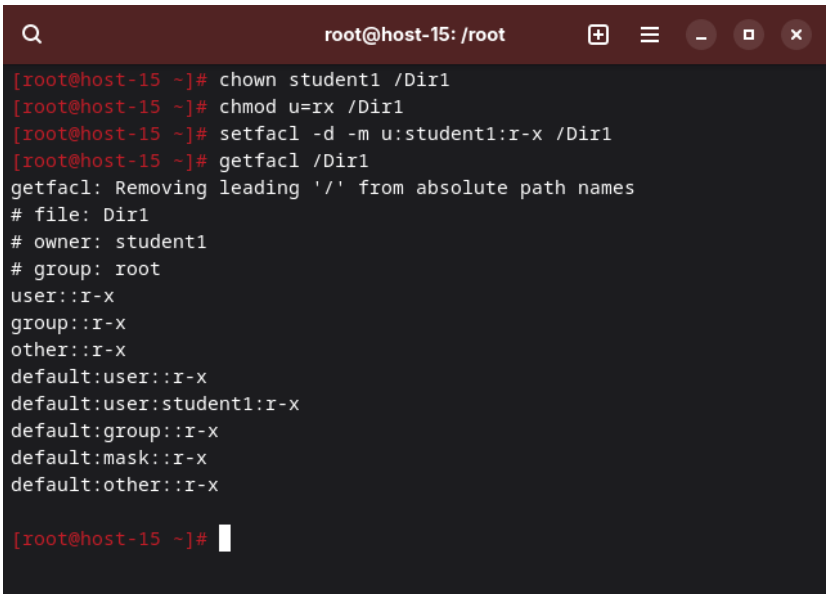


Рис. 11 - Настройка Dir1: владелец student1 и наследуемая запись r-x

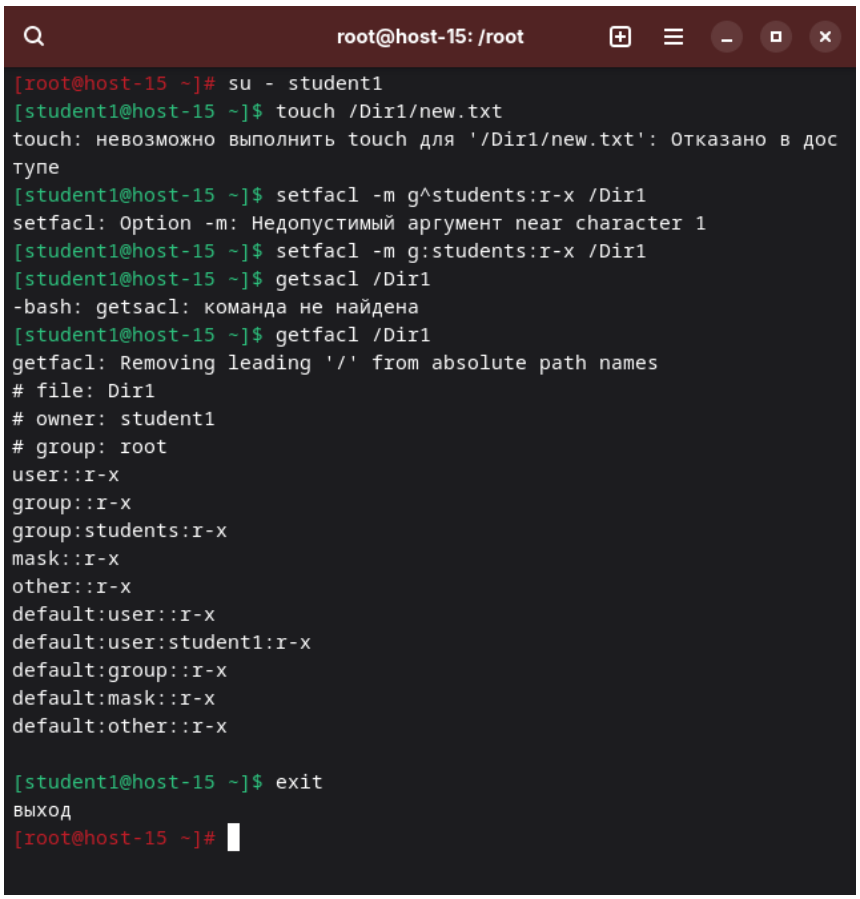


Рис. 12 - student1 не может создать файл в Dir1, но изменяет ACL каталога

Выполнены команды `chown student1 /Dir1` (student1 получает право управлять ACL), `chmod u=rx /Dir1` (у владельца нет права записи) и `setfacl -d -m u:student1:r-x /Dir1`. В выводе `getfacl` видны владелец student1, `user::r-x` и `default:user:student1:r-x`. Проверка от имени student1: создать файл `/Dir1/new.txt` нельзя (нет права w), но команда `setfacl -m g:students:r-x /Dir1` выполнена успешно, и `getfacl` показывает группу students.

Управление ACL в Linux даёт владение каталогом, поэтому student1 назначен владельцем Dir1 (команда `setfacl -m g:students:r-x /Dir1` от его имени выполнилась успешно). Отсутствие права `w` у владельца (`chmod u=rx`) запрещает создавать, удалять и переименовывать файлы в Dir1. Запись `default:user:student1:r-x` без права записи - это то, что получат новые файлы каталога: student1 сможет их читать, но не изменять (право `x` у обычных файлов не появится).

Пункт 2. Наследование в ACL нового файла

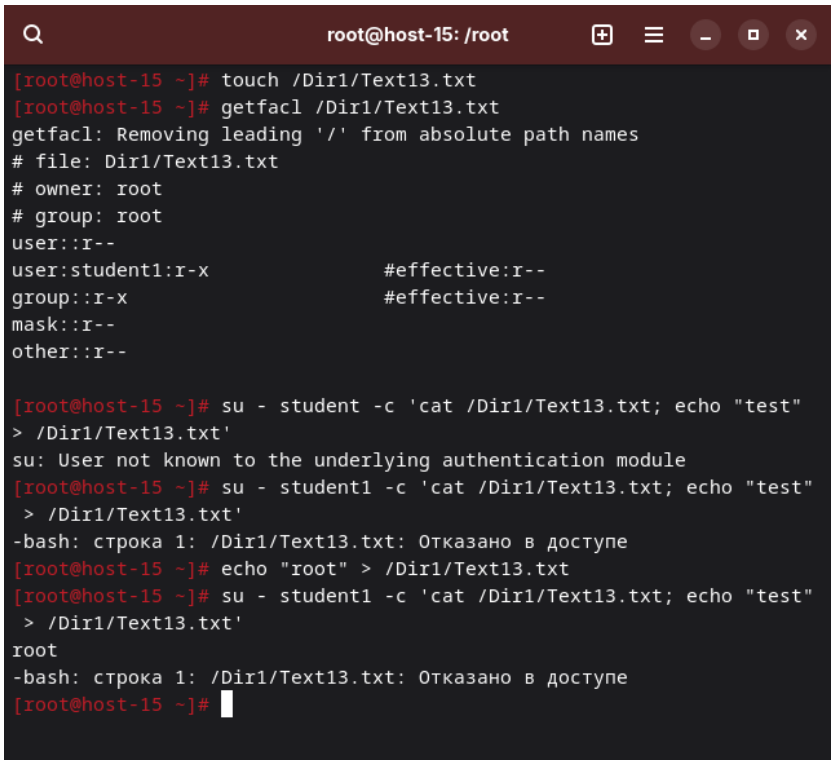


Рис. 13 - Файл Text13.txt получает запись student1:r-x из default ACL каталога

Командой touch /Dir1/Text13.txt создан файл (файл Text1.txt находится в корне, а не в Dir1, поэтому наследование проверялось на новом файле). В его ACL появилась запись user:student1:r-x, скопированная из default ACL каталога, но её действующее право (#effective) равно r--. Причина в том, что при наличии default ACL umask не применяется. Права нового файла определяются как пересечение режима, который запросила программа, и default ACL каталога. Команда touch запрашивает режим 0666 (без x), поэтому получилось mask::r-- (запрошено rw-, в default:mask указано r-x), а x не получил ни один субъект.

Проверка от имени student1: файл был пустым, поэтому cat ничего не вывел, а запись командой echo "test" > /Dir1/Text13.txt завершилась отказом, так как действующее право student1 равно r--. После записи строки «root» от имени root student1 её прочитал, но перезаписать файл не смог. Право x через наследование получить нельзя: его нет в запрошенном режиме, и оно дополнительно ограничено маской.

Я не поняла, что конкретно требовалось в этом пункте, поэтому небольшой разбор наследования в Linux:

Всё строится вокруг -R (рекурсии) и -d (дефолтных или стандартных настроек). Для наглядности я привела все файлы в каталоге /Dir1 к параметрам разрешений ---. После чего командой `setfacl -R -m mask:rwx,other:rwx,u:student1:r-x /Dir1` задала для всех файлов и каталогов внутри /Dir1 (и для самого /Dir1) значения разрешений (Рис. 14 – 15). При этом для новых файлов создаваемых в этой директории значения по умолчанию не поменялись, из-за этого новые файлы не унаследуют значения разрешений. (Рис. 16).

```
[root@host-15 ~]# setfacl -R -m mask:rwx,other:rwx,u:student1:r-x /Dir1
[root@host-15 ~]# getfacl /Dir1
getfacl: Removing leading '/' from absolute path names
# file: Dir1
# owner: root
# group: root
user::rwx
user:student1:r-x
user:teacher:rwx
group::r-x
mask::rwx
other::rwx
default:user::rwx
default:user:student1:---
default:user:teacher:rwx        #effective:---
default:group::r-x              #effective:---
default:mask::---
default:other::---

[root@host-15 ~]# getfacl /Dir1/Dir2
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Dir2
# owner: root
# group: root
user::rwx
user:student1:r-x
user:teacher:rwx
group::r-x
mask::rwx
other::rwx
default:user::rwx
default:user:student1:---
default:group::r-x              #effective:---
default:mask::---
default:other::---
```

Рис. 14 – Установка прав на каталог /Dir1 с использованием -R

```
default:other:---

[root@host-15 ~]# getfacl /Dir1/Text11.txt
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Text11.txt
# owner: root
# group: root
user::rw-
user:student1:r-x
user:teacher:rwx
group::r--
mask::rwx
other::rwx
```

Рис. 15 – Проверка разрешений на файле Text11.txt

```
[root@host-15 ~]# touch /Dir1/Text14.txt
[root@host-15 ~]# getfacl /Dir1/Text14.txt
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Text14.txt
# owner: root
# group: root
user::rw-
user:student1:---
user:teacher:rwx                #effective:---
group::r-x                      #effective:---
mask::---
other::---
```

Рис. 16 – Проверка разрешений новых файлов

Чтобы вновь созданные файлы также обладали необходимыми разрешениями нужно прописать команду `setfacl -R -d -m mask:rwx,other:rwx,u:student1:r-x /Dir1` она устанавливает рекурсивно стандартные разрешения ACL, но при этом не меняет разрешения уже созданных файлов (Рис. 17 - 18), поэтому если необходимо чтобы и уже созданные и новые файлы обладали разрешениями нужно прописать обе команды. (в данном случае важны конкретно комбинации -R и -d, т. к. `mask:rwx,other:rwx,u:student1:r-x` это набор выдаваемых разрешений. Но нужно учитывать, что итоговые разрешения отдельных групп и пользователей зависят от комбинаций разрешений (действующие разрешения показаны в #effective: ---), поэтому необходимо следить не только за теми разрешениями, которые выданы например конкретному пользователю, но и mask и т.д., также разрешение x не выдаётся автоматически для mask и other (по крайней мере командой touch))

```
[root@host-15 ~]# setfacl -R -d -m mask:rwx,other:rwx,u:student1:r-x /Dir1
[root@host-15 ~]# getfacl /Dir1/Dir2
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Dir2
# owner: root
# group: root
user::rwx
user:student1:r-x
user:teacher:rwx
group::r-x
mask::rwx
other::rwx
default:user::rwx
default:user:student1:r-x
default:group::r-x
default:mask::rwx
default:other::rwx

[root@host-15 ~]# getfacl /Dir1
getfacl: Removing leading '/' from absolute path names
# file: Dir1
# owner: root
# group: root
user::rwx
user:student1:r-x
user:teacher:rwx
group::r-x
mask::rwx
other::rwx
default:user::rwx
default:user:student1:r-x
default:user:teacher:rwx
default:group::r-x
default:mask::rwx
default:other::rwx
```

Рис. 17 – установка default разрешений

```
[root@host-15 ~]# getfacl /Dir1/Text14.txt
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Text14.txt
# owner: root
# group: root
user::rw-
user:student1:---
user:teacher:rwx                #effective:---
group::r-x                      #effective:---
mask::---
other::---

[root@host-15 ~]# touch /Dir1/Text15.txt
[root@host-15 ~]# getfacl /Dir1/Text15.txt
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Text15.txt
# owner: root
# group: root
user::rw-
user:student1:r-x               #effective:r--
user:teacher:rwx                #effective:rw-
group::r-x                      #effective:r--
mask::rw-
other::rw-
```

Рис. 18 – проверка ранее созданного файла Text14.txt и нового файла Text15.txt

После все новые файлы, созданные для демонстрации были удалены и разрешения возвращены к тем, которые были до разбора наследования.

Невозможность изменения унаследованного назначения и работа назначения. В NTFS унаследованную запись нельзя изменить на дочернем объекте, даже администратору: она привязана к родителю. В Linux это не так: от имени root выполнена команда `setfacl -m u:student1:rw /Dir1/Text13.txt` (рис. 19), и она прошла без ошибки, запись стала `user:student1:rw-`. Причина в том, что default ACL лишь копируется в файл в момент создания (источник 7); после этого запись ничем не отличается от обычной и не связана с родителем, а root к тому же не подчиняется проверкам прав (владелец файла также может менять его ACL). Запретить такое изменение средствами POSIX ACL нельзя. Работа назначения проверена выше: пока в ACL было `user:student1:r-x` (действующее `r--`), student1 читал файл, но записать в него не мог.

Пункт 3. Разрешение на изменение содержимого

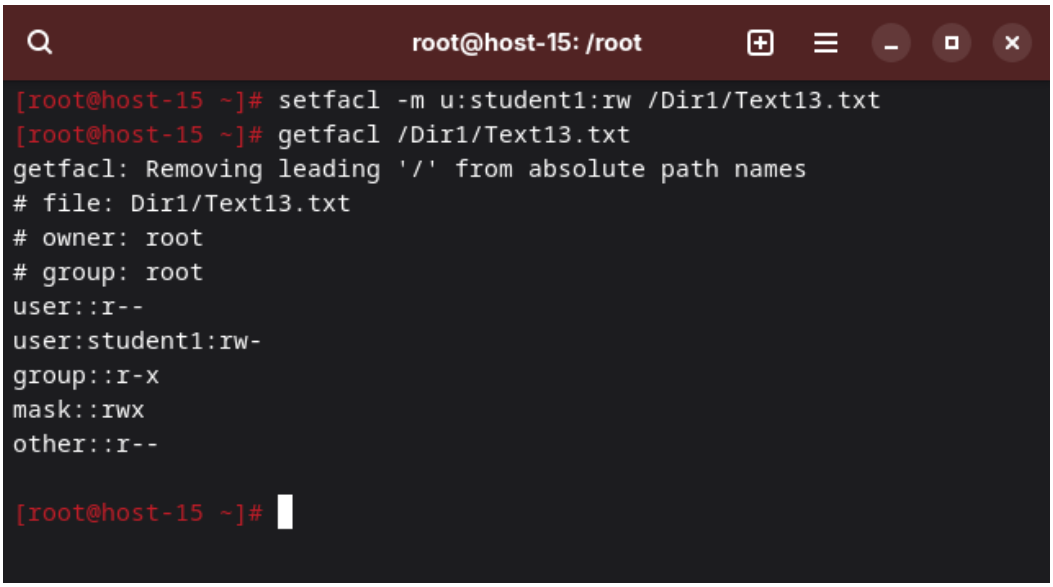


Рис. 19 - Изменение прав student1 на Text13.txt

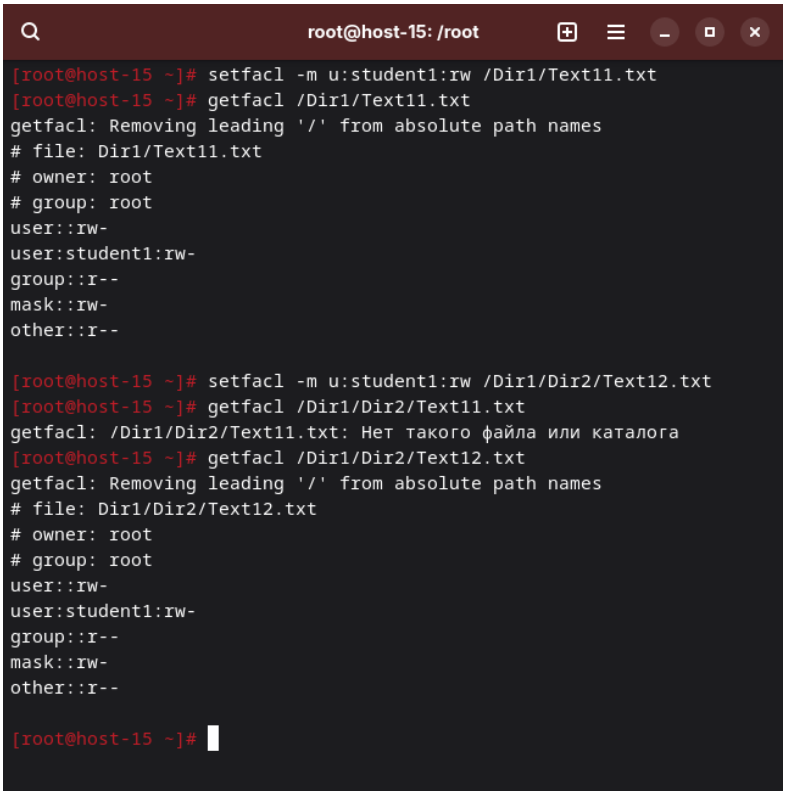


Рис. 20 - Назначение student1 прав rw на Text11.txt и Text12.txt

Для унаследованного файла выполнено `setfacl -m u:student1:rw /Dir1/Text13.txt`: запись стала `user:student1:rw-`, маска выросла до `mask::rwx`. Те же права назначены Text11.txt и Dir2/Text12.txt, и `getfacl` показывает `user:student1:rw-` и `mask::rw-`. Изменение сразу видно в выводе `getfacl`.

Базовое представление (`ls -l`): права Text11.txt до назначения были `-rw-r--r--` (рис. 2), после назначения стали `-rw-rw-r--+` (рис. 22): появился знак «+» (Знак + в конце строки прав означает, что у файла есть расширенный ACL, то есть дополнительные записи для отдельных пользователей или групп сверх обычных трёх наборов прав), а в средней тройке показаны не права группы, а значение маски. Расширенное представление (`getfacl`) показывает отдельные записи `user:student1:rw-`, `group::r--` и `mask::rw-` (рис. 20). Для Text12.txt получен тот же результат.

Пункт 4. Источник назначений

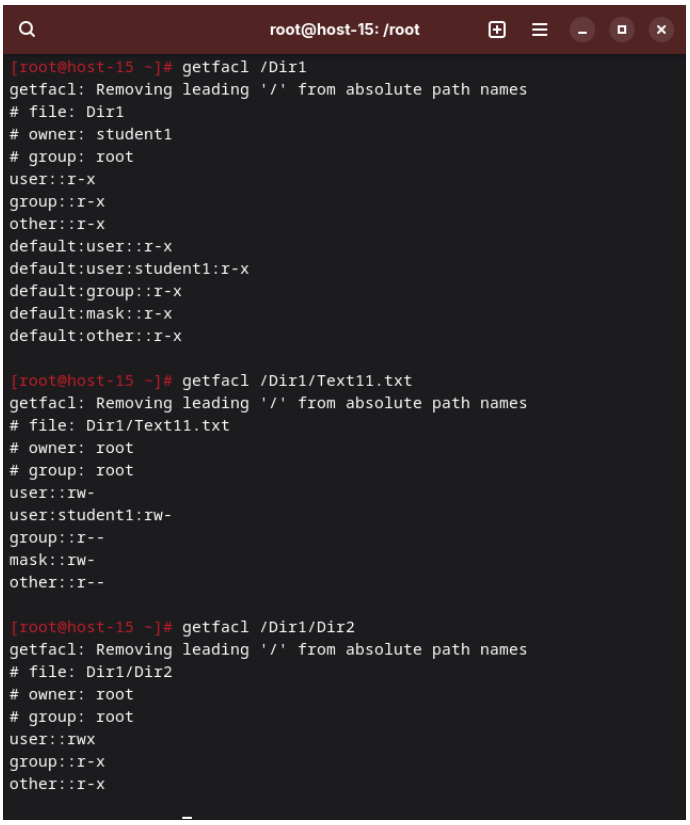


Рис. 21 - ACL каталога Dir1, файла Text11.txt и каталога Dir2

Столбца «Унаследовано от» в Linux нет, источник определяется по default-записям родителя (рис. 21). В Dir1 заданы `default:user:student1:r-x` и остальные default-записи, но Text11.txt создан раньше, чем появился default ACL, поэтому ничего не унаследовал. Запись `user:student1:rw-` в нём явная, она задана вручную в пункте 3. У Dir2 расширенных записей вообще нет: только базовые права `rwxr-xr-x`. Наследование касается лишь объектов, созданных после появления default ACL. (поскольку к данным элементам не применялось рекурсивное задание прав, действие которого описано в пункте 2)

Пункт 5. Действующие разрешения

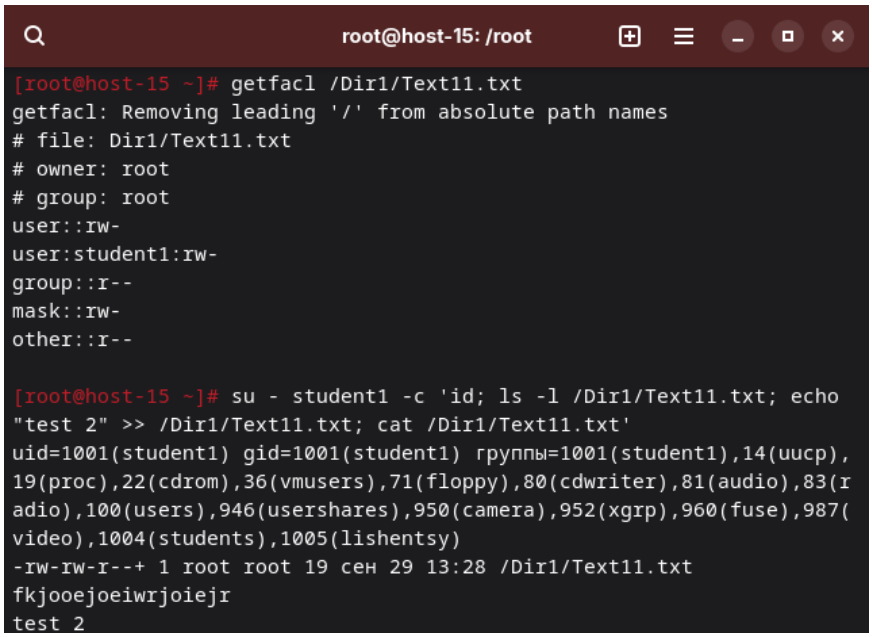


Рис. 22 - Определение действующих прав student1 на Text11.txt

В `getfacl` для файла указаны `user:student1:rw-` и `mask::rw-`, значит действующие права - чтение и запись. `ls -l` показывает `-rw-rw-r--+`: знак «+» означает наличие ACL, а средняя тройка символов отражает маску. От имени student1 выполнены `id`, `ls -l` и дописывание строки, которое прошло успешно. Соответственно student1 имеет на Text11.txt права чтения и записи без выполнения (rw-).

Упражнение 4 - Запрет наследования, восстановление доступа к файлу

Пункт 6. Файл Text13.txt от имени student2

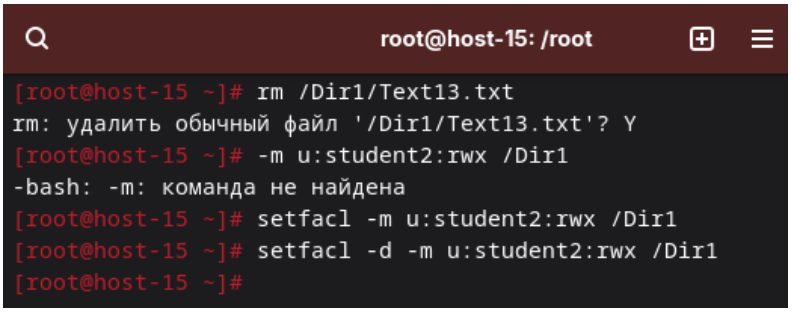


Рис. 23 - Удаление старого Text13.txt и выдача student2 прав на Dir1

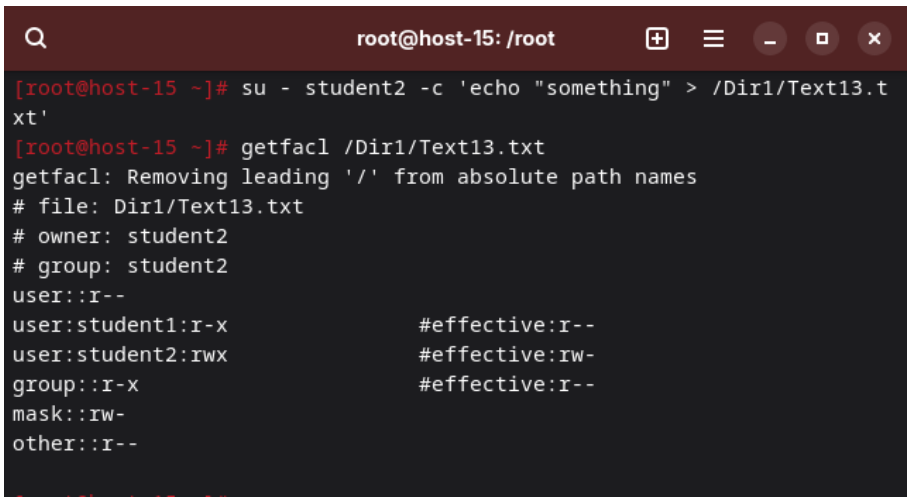


Рис. 24 - Новый Text13.txt, созданный student2, и его ACL

Сначала удалён файл Text13.txt, созданный ранее от root (рис. 23), и student2 получил права rwx на Dir1 в обычной и default-записи: `setfacl -m u:student2:rwx /Dir1` и `setfacl -d -m u:student2:rwx /Dir1`. Затем файл создан командой `su - student2 -c 'echo "something" > /Dir1/Text13.txt'`. Владелец и группа файла - student2, а его ACL содержит записи `user:student1:r-x` и `user:student2:rwx`, полностью совпадающие с default-записями Dir1.

Права файла при создании ограничены режимом создания. Запись владельца `user::r--` также получена из `default:user::r-x` каталога. По порядку проверки ACL для владельца действует именно запись `user::`, а не именованная запись `user:student2:rwx`, поэтому действующие права student2 на собственный файл определяет `user::`.

Пункт 7. Отключение наследования с копированием

у файла в Linux нет флага наследования: default ACL копируется в объект один раз при создании. Поэтому ACL файла будет таким, какой был сформирован при его создании, пока его не изменить на прямую или косвенно.

Пункт 8. Teacher в ACL каталога

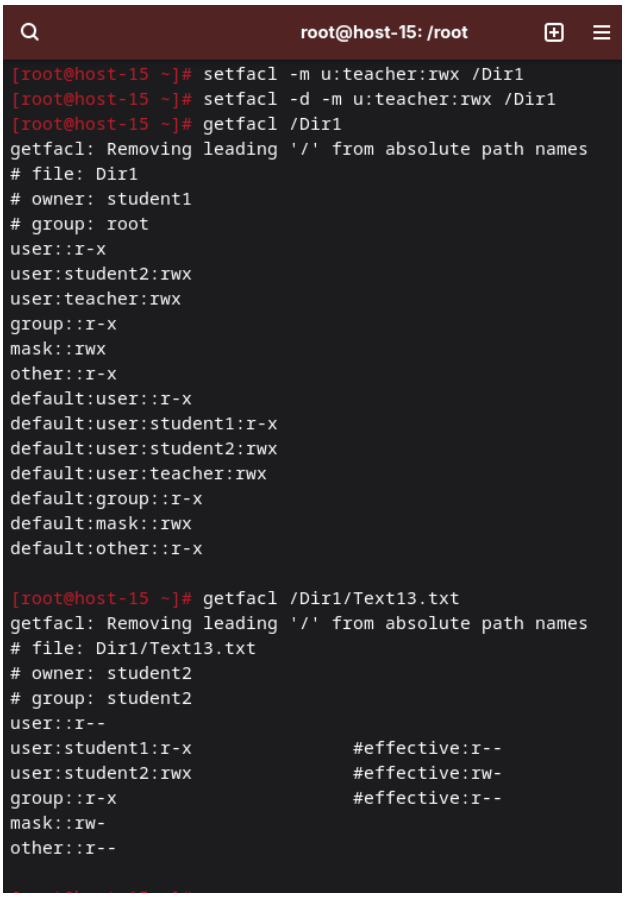


Рис. 25 - Добавление teacher в ACL Dir1 и ACL файла Text13.txt

Выполнены `setfacl -m u:teacher:rwx /Dir1` и `setfacl -d -m u:teacher:rwx /Dir1`. В ACL Dir1 появились `user:teacher:rwx` и `default:user:teacher:rwx`, а в ACL уже существующего Text13.txt записи teacher нет. После создания файл не связан с каталогом, поэтому изменения родителя на него не распространяются.

Пункты 9-10. Преобразование и очистка ACL

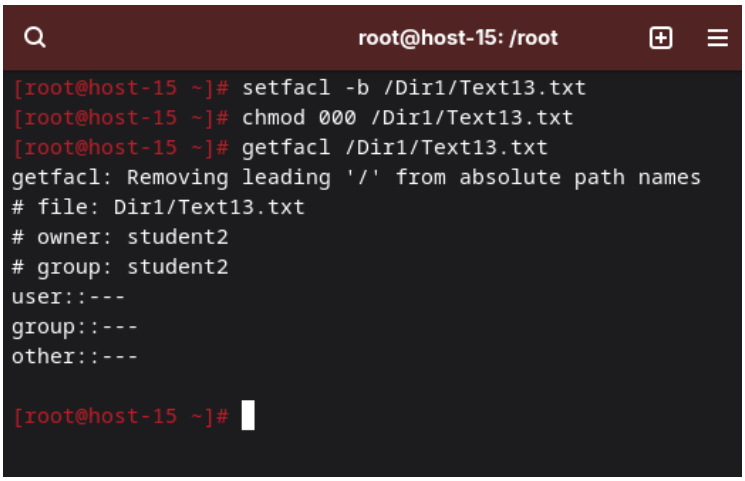


Рис. 26 - Очистка ACL файла Text13.txt

Преобразование не требуется, тк записи уже явные (пункт 7). Для очистки выполнены `setfacl -b /Dir1/Text13.txt` (удалены все расширенные записи) и `chmod 000 /Dir1/Text13.txt`. `getfacl` показывает только `user::---`, `group::---`, `other::---`. Полностью пустым ACL в Linux быть не может, потому что три базовые записи удалить нельзя, поэтому «пустой список» достигается обнулением прав.

Пункты 11-12. Доступ к файлу без записей ACL

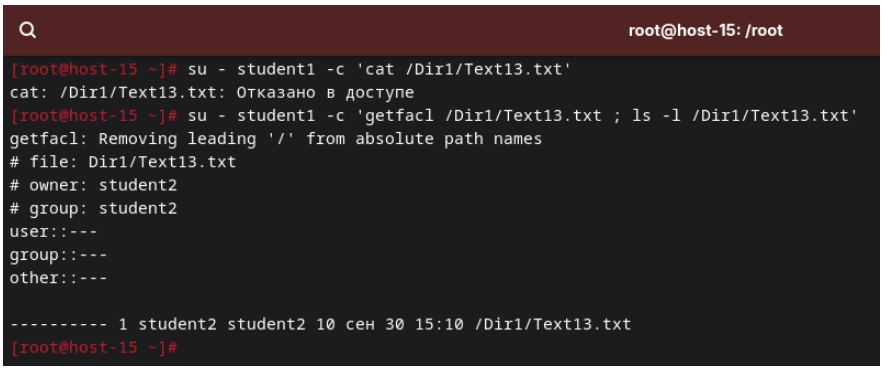


Рис. 27 - Попытка student1 прочитать Text13.txt и получить его ACL

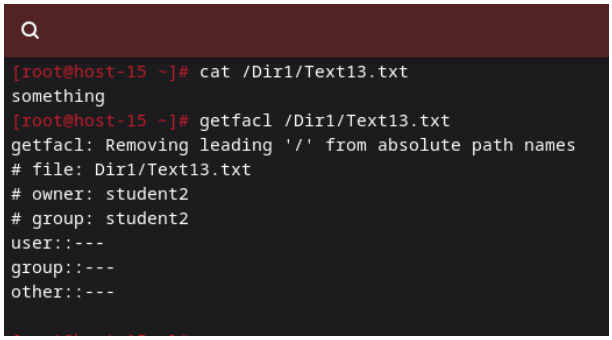


Рис. 28 - Чтение Text13.txt и просмотр ACL от имени root

student1 не смог прочитать файл («Отказано в доступе»), но `getfacl` и `ls -l` от его имени сработали: просмотр ACL требует лишь права x на каталоги пути, а не прав на сам файл. root прочитал файл и просмотрел ACL, так как привилегированный пользователь не проверяется на права доступа к файлам. В Windows пустой ACL закрывает файл и для администратора, пока он не станет владельцем; в Linux такой блокировки root нет.

Пункт 13. Файл Text2.txt

Не понятно откуда должен был взяться файл Text2.txt, и что этот пункт должен показывать, поэтому он фактически пропущен, однако файл Text2.txt создан в каталоге /Dir1 от имени root.

Пункт 14. Смена владельца

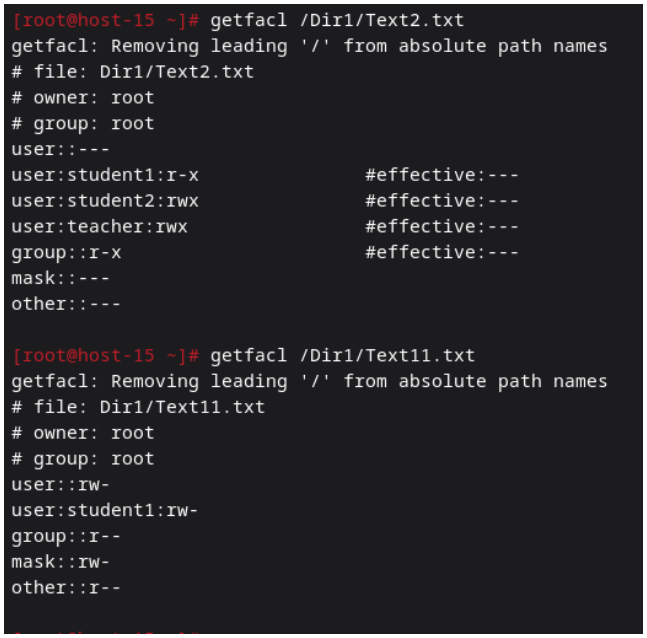


Рис. 29 - Text2.txt и ACL файла Text11.txt

Я также не совсем понимаю, что требовалось выяснить в данном пункте, причём тут файл Text11.txt и кому до этого принадлежал Text2.txt по задумке методички. Несмотря на это, если права администратора на этот файл были ограничены, то он может сам их себе выдать и стать его владельцем. На Text11.txt это никак не повлияет (либо не хватает каких-то важных вводных данных про эти файлы)

Пункт 15. Те же операции для каталога Dir1

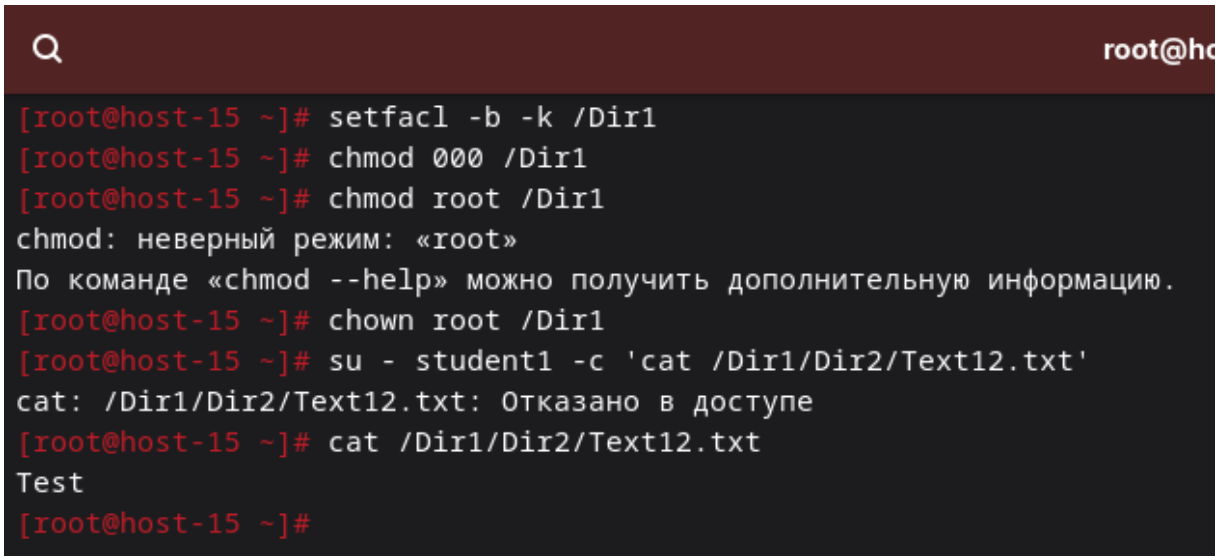


Рис. 30 - Снятие ACL и прав с каталога Dir1, проверка доступа к Text12.txt

Выполнены `setfacl -b -k /Dir1` (удалены обычные и default ACL), `chmod 000 /Dir1` и `chown root /Dir1`. Проверка: student1 не смог прочитать `/Dir1/Dir2/Text12.txt`, хотя на сам файл у него есть права. Без права x на Dir1 войти в каталог нельзя, а при обращении по пути проверяется доступ ко всем каталогам цепочки. root прочитал файл без ограничений.

Пункт 16. Повторное включение наследования

```
[root@host-15 ~]# chmod 755 /Dir1
[root@host-15 ~]# setfacl -d -m u:student1:r-x /Dir1; setfacl -d -m u:teacher:rwx /Dir1
[root@host-15 ~]# getfacl /Dir1/Text11.txt
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Text11.txt
# owner: root
# group: root
user::rw-
user:student1:rw-
group::r--
mask::rw-
other::r--

[root@host-15 ~]# setfacl -R -m u:teacher:rwx /Dir1
[root@host-15 ~]# getfacl /Dir1/Text11.txt
getfacl: Removing leading '/' from absolute path names
# file: Dir1/Text11.txt
# owner: root
# group: root
user::rw-
user:student1:rw-
user:teacher:rwx
group::r--
mask::rwx
other::r--
```

Рис. 31 - Восстановление default ACL каталога Dir1 и применение teacher рекурсивно

Доступ к каталогу возвращён командой `chmod 755 /Dir1`, default ACL восстановлен командами `setfacl -d -m u:student1:r-x /Dir1` и `setfacl -d -m u:teacher:rwx /Dir1`. После этого в ACL существующего Text11.txt запись teacher не появилась: default ACL применяется только к новым объектам. Чтобы распространить права на существующие, выполнена команда `setfacl -R -m u:teacher:rwx /Dir1`: теперь в Text11.txt есть `user:teacher:rwx`, маска стала `mask::rwx`.

После этой операции владельцем каталога Dir1 остался root, а default-запись для student2 не восстанавливалась: при необходимости их можно вернуть командами `chown student1 /Dir1` и `setfacl -d -m u:student2:rwx /Dir1`.

Выводы.

1. В ALT Linux расширенные права реализованы списками POSIX ACL: к базовым правам владельца, группы и остальных добавляются записи отдельных пользователей и групп. Это позволяет выдать доступ точечно, не меняя владельца и группу файла. Наличие ACL видно по знаку «+» в выводе ls -l, а подробности - в getfacl.
2. Эксперимент с группами students (rwx) и lishentsy (r--) показал, что права суммируются: student1, состоящий в обеих группах, продолжил писать в файл. Ограничить доступ можно только тем, что разрешающей записи нет, или персональной записью пользователя, которая проверяется раньше групповых.
3. Управление ACL нельзя отделить от владения: менять ACL может только владелец или root. Владелец файла с правами 000 самостоятельно вернул себе доступ командой setfacl, поэтому передача владения равносильна передаче полного контроля над правами файла.
4. Действующие права определяет маска. Команда chmod на файле с ACL меняет маску, а не групповую запись, и обнуляет действующие права всех именованных записей, хотя сами записи остаются на месте. При диагностике доступа надо смотреть поле #effective в getfacl, а не только перечень записей.
5. Доступ к файлу зависит от всей цепочки каталогов: для входа нужно право x на каждом каталоге пути, а удаление и создание файлов определяются правом w на каталоге. Каталог Dir1 с правами 000 закрыл доступ к Text12.txt, хотя права на самом файле были выданы.
6. Наследование через default ACL одноразовое: записи копируются в объект при создании, после чего он независим от родителя. Изменение default ACL не затрагивает существующие объекты, поэтому «отключать» наследование не нужно, а чтобы применить новые права к имеющимся файлам, нужен рекурсивный запуск setfacl -R. Столбца «Унаследовано от» нет, происхождение записи определяется

сравнением с default ACL родителя и датой создания объекта. Унаследованную запись нельзя защитить от изменения: после копирования её может менять и владелец, и root.

7. Пользователь root обходит проверки прав: он читал файлы с нулевыми правами и с пустым набором записей, а смена владельца для него не нужна.

Источники:

1. https://www.reddit.com/r/linuxquestions/comments/4oynfc/what_does_rwrr_mean/?tl=ru
2. https://firstvds.ru/technology/linux-permissions
3. https://wiki.astralinux.ru/plugins/viewsource/viewpagesrc.action?pageId=137567951
4. https://gitverse.ru/blog/articles/development/352-prava-polzovatelya-v-linux
5. https://help.sweb.ru/acl-v-linux-nastrojka-prav-dostupa_1521.html
6. https://unix.stackexchange.com/questions/193986/who-can-change-acl-permissions
7. https://man7.org/linux/man-pages/man5/acl.5.html